Local Execution Guide

Mortgage Pricing Exceptions | Docker-first setup and verification

1. Prerequisites

Install Git and Docker Desktop (Linux containers). Use a web browser. Internet access is needed for the first download/build. For Docker-only execution, no separate .NET, Node.js, or PostgreSQL installation is required.

Clone the supplied Git repository and open its folder in VS Code or PowerShell. Run all Docker commands from the repository root (the folder containing compose.yaml).

```
git clone <repository-url>
cd <repository-folder>
```

2. Start the application (recommended)

Start Docker Desktop, then run:

```
docker compose up -d --build
docker compose ps
```

Wait for the services to start. The stack includes PostgreSQL, ASP.NET Core API, and React frontend. On startup, the API applies EF Core migrations and initializes development demo users.

Open	Local address
Frontend	http://localhost:5173
Swagger API	http://localhost:5267/swagger

Default ports are 5173 (frontend), 5267 (API), and 5432 (database). Ensure they are free. Do not run host and Docker versions simultaneously on the same ports.

3. Verify the business workflow

In the frontend, select James Wilson (Manager). Create a pricing exception using any one of the sample mortgage application IDs below, select David Miller as assigned reviewer, enter the discount and reason, and submit for review. Switch to David Miller (Reviewer) and approve or decline. For an approved request, select Demo Mortgage Process and look up the same mortgage application ID. Open request history to inspect decisions and revisions.

Understanding BPS (basis points)

BPS means basis points (not “base points”). One basis point equals 0.01 percentage points; 100 BPS = 1 percentage point. Example: a 50 BPS approved discount reduces a 5.00% mortgage rate to 4.50%. The application accepts discounts from 1 to 500 BPS.

Copy-ready sample test data

These are synthetic mortgage application GUIDs, not user IDs. Use one unique ID per new request. Requester: James Wilson (Manager); assigned reviewer: David Miller (Reviewer).

#	Mortgage application ID	BPS	Supporting reason

1	a1111111-1111-4111-8111-111111111111	50	Customer retention; competitive pricing
2	a2222222-2222-4222-8222-222222222222	75	Competing lender offered a lower rate
3	a3333333-3333-4333-8333-333333333333	100	Long-term customer preferential rate
4	a4444444-4444-4444-8444-444444444444	25	Existing relationship pricing adjustment
5	a5555555-5555-4555-8555-555555555555	125	High-value customer retention

Example development user ID: James Wilson = 11111111-1111-1111-1111-111111111111 (as shown in the UI). This identifies the requester, NOT the mortgage application. For direct API calls, use a seeded identity in the X-User-Id header in Development; the frontend lets you select users by name.

Revision check: after approval, switch back to James Wilson, create and submit a revised 75 BPS proposal for the same mortgage application, then inspect the request status and history. Verify approved-discount lookup behavior against the application's defined revision rules.

4. Run automated tests (optional)

Additional prerequisites: .NET 10 SDK for backend tests; Node.js 24/npm and Microsoft Edge for frontend/browser tests. Keep Docker running for database-backed tests. Each command block below starts from the repository root in a fresh PowerShell terminal.

Backend: unit, integration and PostgreSQL tests

```
cd Backend/BlueHarvest.PricingExceptions
dotnet test
```

Expected: the solution test projects run and report their results; investigate any failures.

Frontend: build, lint and Playwright end-to-end tests

```
cd FrontEnd
npm.cmd ci
npm.cmd run build
npm.cmd run lint
npm.cmd run test:e2e
```

Leave E2E_BASE_URL unset for the isolated browser tests, which use ports 5174 and 5270.

API smoke tests

With the full Docker stack running, execute from the repository root:

```
powershell.exe -NoProfile -ExecutionPolicy Bypass `
  -File Backend/BlueHarvest.PricingExceptions/tests/Invoke-ApiSmokeTests.ps1 `
  -BaseUrl http://localhost:5173
```

Expected: successful checks and zero exit code. Smoke tests may create demo records.

5. Stop, restart and troubleshoot

```
docker compose stop          # Stop; keep database data
docker compose up -d         # Restart existing containers
```

```
docker compose ps              # Inspect container status
docker compose logs --tail=100 # Inspect startup/errors
```

The named database volume preserves data. Do not run docker compose down -v unless you intend to delete the database. If startup fails, confirm Docker Desktop is running, inspect logs, and check for port conflicts.

Note: Commands and expected behavior reflect the supplied project configuration. Verify against the final repository checkout before external handoff.